



Java socket docs

☰ Tags	OP
# Cap	-99
⚙ Status	Done
🕒 Ultima modifica	@February 11, 2024 3:16 PM

Info e consigli

Generici

Socket Stream

Costruttori client

Costruttori server

La accept (server)

Informazioni sulla socket

Stream di comunicazione

Comunicazione a stream

String

Char

Int

Float

Gestione delle eccezioni

Chiusura delle socket

Esempio

Info e consigli

Generici

- Fare i controlli per i parametri in ingresso ovunque
- Non usare funzioni
- Non usare struct e/o classi
- Tutte le variabili vanno inizializzate e istanziate al di fuori dei cicli

- Utilizzare i tipi che occupano meno spazio possibile
-

Socket Stream

Costruttori client

- `public Socket(InetAddress remoteHost, int remotePort) throws IOException`
- `public Socket (String remoteHost, int remotePort) throws IOException`
- `public Socket(InetAddress remoteHost, int remotePort, InetAddress localhost, int localPort) throws IOException;`

Costruttori server

- `public ServerSocket(int localPort) throws IOException, BindException`
crea una socket in ascolto sulla porta specificata
- `public ServerSocket(int localPort, int count)`
crea una socket in ascolto sulla porta specificata con una coda di lunghezza count

La accept (server)



Bloccante

- `public Socket accept() throws IOException;`

Informazioni sulla socket

- `public InetAddress getInetAddress();`
(restituisce l'indirizzo del nodo remoto a cui socket è connessa)
- `public InetAddress getLocalAddress();`
(restituisce l'indirizzo della macchina locale)

- `public int getPort();`
(restituisce il numero di porta sul nodo remoto cui socket è connessa)
- `public int getLocalPort();`
(restituisce il numero di porta locale a cui la socket è legata)

Stream di comunicazione

- `public InputStream getInputStream();`
- `public OutputStream getOutputStream();`



I due metodi restituiscono un oggetto stream che incapsula il canale di comunicazione (di classi `InputStream` e `OutputStream`)



Attraverso gli stream (generici di byte) si possono spedire/ricevere solo byte, senza nessuna formattazione in messaggi (vedi classi)

Comunicazione a stream

String

- `void writeUTF(String str)` : Scrive una stringa UTF nel flusso di output.
- `String readUTF()` : Legge una stringa UTF dal flusso di input.

Char

- `void writeChar(int v)` : Scrive un carattere nel flusso di output.
- `char readChar()` : Legge un carattere dal flusso di input.

Int

- `void writeInt(int v)` : Scrive un intero nel flusso di output.
- `int readInt()` : Legge un intero dal flusso di input.

Float

- `void writeFloat(float v)` : Scrive un numero in virgola mobile nel flusso di output.

- `float readFloat()` : Legge un numero in virgola mobile dal flusso di input.

Gestione delle eccezioni

Ogni metodo che coinvolge operazioni di I/O può generare eccezioni come `IOException`. È importante gestire queste eccezioni in modo appropriato per garantire la stabilità e l'affidabilità dell'applicazione.

Chiusura delle socket

Per liberare le risorse e chiudere correttamente una socket, è necessario chiamare il metodo `close()` sulla socket stessa. È buona pratica chiudere sia la socket client che quella del server dopo che la comunicazione è terminata.

```
// Chiusura della socket client
clientSocket.close();

// Chiusura della socket del server
serverSocket.close();
```

Esempio

```
// Esempio di utilizzo del client Socket
try {
    Socket clientSocket = new Socket("localhost", 8080);
    OutputStream outputStream = clientSocket.getOutputStream();
    outputStream.writeUTF("Hello, Server!");

    InputStream inputStream = clientSocket.getInputStream();
    String response = inputStream.readUTF();
    System.out.println("Server response: " + response);

    clientSocket.close();
}
```

```
} catch (IOException e) {  
    e.printStackTrace();  
}  
  
// Esempio di utilizzo del server ServerSocket  
try {  
    ServerSocket serverSocket = new ServerSocket(8080);  
    Socket clientSocket = serverSocket.accept();  
  
    InputStream inputStream = clientSocket.getInputStream();  
    String request = inputStream.readUTF();  
    System.out.println("Client request: " + request);  
  
    OutputStream outputStream = clientSocket.getOutputStream();  
    outputStream.writeUTF("Hello, Client!");  
  
    clientSocket.close();  
    serverSocket.close();  
} catch (IOException e) {  
    e.printStackTrace();  
}
```